

SSO & Access Management Implementation Plan

Azure AD (Entra ID) single sign-on with role- and team-based feature gatekeeping

Stack	React (frontend) · FastAPI (backend) · PostgreSQL
Hosting	AWS
Identity provider	Azure Active Directory / Microsoft Entra ID
Auth protocol	OpenID Connect (OIDC), Authorization Code + PKCE
Scope	SSO login, group-to-role/team mapping, feature-level gatekeeping

1. Executive summary

This document specifies how to add Azure AD single sign-on to the application and use it to drive feature-level access control. The design has three layers that are deliberately kept independent:

- **Authentication** — Azure AD proves who the user is via OIDC and returns their group memberships as token claims.
- **Authorization mapping** — a small set of tables in our own database translate Azure AD groups into application Roles (what a user can do) and Teams (what data/workspace a user belongs to). This mapping is owned and editable by us, independent of how IT structures Azure AD groups.
- **Enforcement** — FastAPI dependencies check the resolved role and team against a feature permission matrix on every request. The React frontend mirrors these checks for UX only; the backend is the actual security boundary.

The initial gatekept features are **Form Factory**, **Upload Contacts**, and **Connected Apps**, each restricted by a combination of role (Viewer / Editor / Team Admin / Org Admin) and team membership. The pattern is designed to extend to new features by adding rows to a permissions table — no code changes required to gate a new feature to an existing role.

1.1 Guiding principles

- **Never gate on raw Azure AD group membership in code.** Always go through the internal Role/Team mapping so group renames, restructures, or IT-side changes don't require an application deploy.
- **Enforce on the backend, mirror on the frontend.** The React UI hides/disables gated features for good UX, but every API route independently re-checks permission. Client-side checks are never trusted.
- **Deny by default.** A user whose Azure AD groups don't map to any known role gets no feature access, not implicit access.
- **Data-driven permissions.** Role → feature access lives in a database table, not in scattered `if` statements.

2. Architecture overview

End-to-end flow: the user authenticates against Azure AD from the React SPA, the SPA obtains tokens via MSAL.js, and every subsequent API call to FastAPI carries a bearer access token. FastAPI validates the token against Azure AD's signing keys, resolves the user's groups to internal roles/teams (cached after first resolution), and gates each route through a permission dependency.

2.1 Component diagram (text form)

```
React SPA (MSAL.js)
|   Authorization Code + PKCE
v
Azure AD / Entra ID -----> Microsoft Graph API
|   ID token + access token      (group membership, overage lookups)
v
AWS Application Load Balancer (HTTPS, ACM cert)
|
v
FastAPI backend (ECS Fargate / App Runner)
|-- verify_token dependency      (validates JWT against Azure AD JWKS)
|-- resolve_identity dependency  (loads/creates user, resolves roles+teams)
|-- require_permission dependency (checks role+team vs feature)
v
PostgreSQL (RDS)                 Redis / ElastiCache (optional)
- users, teams, roles, features   - caches resolved role/team per session
- group_role_map, group_team_map  - short TTL, avoids re-querying Postgres
- role_permissions
```

2.2 AWS component choices

Component	Recommendation	Why
Frontend hosting	S3 + CloudFront	Static React build, HTTPS via ACM, cheap and simple to invalidate on deploy.
Backend compute	ECS Fargate (or App Runner for less ops)	FastAPI as a container; Fargate avoids managing EC2 instances.
Load balancing / TLS	Application Load Balancer	Terminates HTTPS, routes to FastAPI tasks, health checks.
Database	RDS PostgreSQL	Stores users/teams/roles/permissions/mappings; Multi-AZ for prod.
Secrets	AWS Secrets Manager	Azure AD client secret/certificate, DB credentials, JWT signing keys if any.
Session cache (optional)	ElastiCache for Redis	Cache resolved role/team per user to avoid DB hit on every request.

Component	Recommendation	Why
Audit logs	CloudWatch Logs + optional S3 export	Login events, permission-denied events, admin mapping changes.

2.3 Why not Cognito

Amazon Cognito could sit as a broker in front of Azure AD (Cognito user pool federating to Azure AD as a SAML/OIDC IdP). This is worth considering later if you need to federate multiple identity providers behind one interface, but it adds an extra hop and another system to configure. Since Azure AD is the only IdP today, integrating FastAPI directly with Azure AD via OIDC is simpler and has one fewer moving part to debug.

3. Azure AD (Entra ID) setup

3.1 App registrations

Create two app registrations in Entra ID:

- **SPA registration** (React) — Platform: Single-page application. Redirect URI: your CloudFront domain (e.g. `https://app.yourcompany.com`). No client secret (public client, PKCE only).
- **API registration** (FastAPI) — Platform: Web API. Expose a scope, e.g. `api://<api-client-id>/access_as_user`. The SPA registration requests this scope so access tokens are issued for your API's audience, not just Microsoft Graph.

3.2 Token configuration — groups claim

On the API app registration, add the groups claim under Token configuration for both ID and access tokens. Two options:

- **Security groups** — simplest to start with; the token includes group object IDs the user belongs to.
- **App roles (recommended if IT will support it)** — define roles like `FormFactory.Editor` directly on the app registration and assign Azure AD groups to them via Enterprise Applications → App roles. This gives you semantically named roles in the token (`roles` claim) instead of opaque group GUIDs, and avoids some of the group-coverage problem below since app role assignment is coarser.

Groups claim coverage

If a user is a member of more than 200 groups (Azure AD default JWT limit), the token omits the groups list and instead includes a `_claim_names` indicator. Your backend must detect this and call Microsoft Graph (`GET /users/{id}/memberOf`) to fetch the full list. Design the `resolve_identity` dependency to handle this from day one — see Section 5.3.

3.3 API permissions

- `User.Read` (delegated) — basic profile, usually default.
- `GroupMember.Read.All` (delegated or application, admin consent required) — needed only if you fetch group membership via Graph for overage handling.
- Grant admin consent for the tenant so users aren't prompted individually.

3.4 Values to collect for configuration

Value	Where used
Tenant ID	Both SPA (MSAL config) and FastAPI (issuer/JWKS URL)
SPA client ID	MSAL.js configuration in React
API client ID / App ID URI	Token audience validation in FastAPI; scope requested by SPA
API client secret or certificate	Only if backend calls Graph on the app's behalf; store in Secrets Manager

Value	Where used
JWKS URI	<a href="https://login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys">https://login.microsoftonline.com/<tenant-id>/discovery/v2.0/keys — used to validate token signatures

4. Data model

PostgreSQL schema (simplified DDL). This is the source of truth the FastAPI permission dependencies query against.

```
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       TEXT UNIQUE NOT NULL,
  azure_oid   TEXT UNIQUE NOT NULL,      -- Azure AD object id (stable, from 'oid' claim)
  display_name TEXT,
  last_login  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE teams (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name       TEXT UNIQUE NOT NULL
);

CREATE TABLE roles (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name       TEXT UNIQUE NOT NULL      -- 'viewer' | 'editor' | 'team_admin' | 'org_admin'
);

CREATE TABLE features (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  key       TEXT UNIQUE NOT NULL,      -- 'form_factory' | 'upload_contacts' | 'connected_apps'
  name      TEXT NOT NULL
);

CREATE TABLE role_permissions (
  role_id     UUID REFERENCES roles(id),
  feature_id  UUID REFERENCES features(id),
  access_level TEXT NOT NULL,          -- 'read' | 'write' | 'admin'
  PRIMARY KEY (role_id, feature_id)
);

CREATE TABLE user_teams (
  user_id     UUID REFERENCES users(id),
  team_id     UUID REFERENCES teams(id),
  PRIMARY KEY (user_id, team_id)
);

CREATE TABLE user_roles (
  user_id     UUID REFERENCES users(id),
  role_id     UUID REFERENCES roles(id),
  team_id     UUID REFERENCES teams(id),  -- role scoped to a team (nullable = org-wide role)
  PRIMARY KEY (user_id, role_id, team_id)
);

-- Azure AD -> internal mapping, edited via an internal admin UI
```

```

CREATE TABLE azure_ad_groups (
  object_id      TEXT PRIMARY KEY,          -- Azure AD group GUID, NOT the display name
  display_name   TEXT NOT NULL
);

CREATE TABLE group_role_map (
  group_object_id TEXT REFERENCES azure_ad_groups(object_id),
  role_id         UUID REFERENCES roles(id),
  PRIMARY KEY (group_object_id, role_id)
);

CREATE TABLE group_team_map (
  group_object_id TEXT REFERENCES azure_ad_groups(object_id),
  team_id         UUID REFERENCES teams(id),
  PRIMARY KEY (group_object_id, team_id)
);

```

4.1 Why group object IDs, not names

Azure AD group display names can be renamed by IT without notice; object IDs (GUIDs) are permanent. `azure_ad_groups.object_id` is the primary key for exactly this reason — mappings survive a group rename automatically.

4.2 Example permission matrix

Feature	Viewer	Editor	Team Admin	Org Admin
Form factory	—	Read/write, own team	Read/write, own team	Read/write, all teams
Upload contacts	—	Write, own team	Write, own team	Write, all teams
Connected apps	—	—	Admin, own team	Admin, all teams

Team scoping (“own team” vs “all teams”) is applied at the query layer using the user's `team_id(s)` from `user_teams`, not by creating a separate role per team.

5. Backend implementation (FastAPI)

5.1 Dependencies to install

```
pip install fastapi uvicorn[standard] python-jose[cryptography] \
    httpx sqlalchemy asyncpg pydantic-settings redis
```

5.2 verify_token — validate the Azure AD access token

Fetch and cache Azure AD's JWKS, validate signature/issuer/audience/expiry on every request. Cache the JWKS response (e.g. 24h TTL) rather than fetching on every call.

```
# auth/tokens.py
import time
import httpx
from jose import jwt
from fastapi import HTTPException, Request

TENANT_ID = settings.AZURE_TENANT_ID
API_CLIENT_ID = settings.AZURE_API_CLIENT_ID
JWKS_URL = f"https://login.microsoftonline.com/{TENANT_ID}/discovery/v2.0/keys"
ISSUER = f"https://login.microsoftonline.com/{TENANT_ID}/v2.0"

_jwks_cache = {"keys": None, "fetched_at": 0}

async def get_jwks():
    if _jwks_cache["keys"] is None or time.time() - _jwks_cache["fetched_at"] > 86400:
        async with httpx.AsyncClient() as client:
            resp = await client.get(JWKS_URL)
            resp.raise_for_status()
            _jwks_cache["keys"] = resp.json()
            _jwks_cache["fetched_at"] = time.time()
    return _jwks_cache["keys"]

async def verify_token(request: Request) -> dict:
    auth_header = request.headers.get("Authorization", "")
    if not auth_header.startswith("Bearer "):
        raise HTTPException(status_code=401, detail="Missing bearer token")
    token = auth_header.removeprefix("Bearer ")

    jwks = await get_jwks()
    try:
        claims = jwt.decode(
            token,
            jwks,
            algorithms=["RS256"],
            audience=API_CLIENT_ID,
            issuer=ISSUER,
        )
    except jwt.JWTError as e:
        raise HTTPException(status_code=401, detail=f"Invalid token: {e}")

    return claims
```

5.3 resolve_identity — groups to internal roles/teams

Called after `verify_token`. Handles the groups-overage case by falling back to Microsoft Graph, then joins against `group_role_map` / `group_team_map`. Result is cached in Redis (short TTL) keyed by user id, so most requests skip both the Graph call and the DB joins.

```
# auth/identity.py
async def resolve_identity(claims: dict = Depends(verify_token), db=Depends(get_db), cache=Depends(get_cache)) -> Identity:
    oid = claims["oid"]

    cached = await cache.get(f"identity:{oid}")
    if cached:
        return Identity.parse_raw(cached)

    group_ids = claims.get("groups")
    if group_ids is None and "_claim_names" in claims:
        # Overage: token omitted groups, fetch from Graph instead
        group_ids = await fetch_groups_from_graph(claims["oid"], claims["access_token_for_graph"])

    user = await get_or_create_user(db, oid=oid, email=claims.get("preferred_username"))

    roles = await db.fetch_roles_for_groups(group_ids)      # via group_role_map
    teams = await db.fetch_teams_for_groups(group_ids)     # via group_team_map

    identity = Identity(user_id=user.id, email=user.email, roles=roles, teams=teams)
    await cache.set(f"identity:{oid}", identity.json(), ttl=900) # 15 min
    return identity
```

5.4 require_permission — the gatekeeping dependency

A dependency factory: call it with the feature key and required access level, and it returns a dependency FastAPI can inject into any route. This is the box shown as “require_permission dependency” in the gatekeeping diagram.

```
# auth/permissions.py
from fastapi import Depends, HTTPException

def require_permission(feature_key: str, min_level: str = "read"):
    async def _check(identity: Identity = Depends(resolve_identity), db=Depends(get_db)):
        allowed = await db.check_permission(
            role_ids=[r.id for r in identity.roles],
            feature_key=feature_key,
            min_level=min_level,
        )
        if not allowed:
            raise HTTPException(status_code=403, detail=f"Not permitted: {feature_key}")
        return identity
    return _check
```

5.5 Using it on a route

```

# routers/form_factory.py
from fastapi import APIRouter, Depends
from auth.permissions import require_permission

router = APIRouter()

@router.post("/api/form-factory")
async def create_form(
    payload: FormCreate,
    identity: Identity = Depends(require_permission("form_factory", "write")),
):
    # identity.teams is already resolved -> scope the write to the user's team(s)
    return await forms_service.create(payload, team_ids=[t.id for t in identity.teams])

@router.post("/api/connected-apps")
async def create_connection(
    payload: ConnectionCreate,
    identity: Identity = Depends(require_permission("connected_apps", "admin")),
):
    return await connections_service.create(payload, team_ids=[t.id for t in identity.teams])

```

Every protected route follows this same one-line pattern. Adding a new gated feature means adding a row to features and role_permissions — not writing new authorization code.

5.6 Expose an /api/me endpoint

The frontend needs to know what to show/hide without guessing. Return the resolved identity and a flattened permission map once at login.

```

@router.get("/api/me")
async def me(identity: Identity = Depends(resolve_identity), db=Depends(get_db)):
    permissions = await db.get_permission_map(role_ids=[r.id for r in identity.roles])
    return {
        "email": identity.email,
        "teams": [t.name for t in identity.teams],
        "roles": [r.name for r in identity.roles],
        "permissions": permissions, # e.g. {"form_factory": "write", "connected_apps":
"admin"}
    }

```

6. Frontend implementation (React)

6.1 Dependencies

```
npm install @azure/msal-browser @azure/msal-react
```

6.2 MSAL configuration

```
// authConfig.js
export const msalConfig = {
  auth: {
    clientId: process.env.REACT_APP_AZURE_SPA_CLIENT_ID,
    authority: `https://login.microsoftonline.com/${process.env.REACT_APP_AZURE_TENANT_ID}`,
    redirectUri: process.env.REACT_APP_REDIRECT_URI, // e.g. https://app.yourcompany.com
  },
  cache: { cacheLocation: "sessionStorage" },
};

export const apiRequest = {
  scopes: [`api://${process.env.REACT_APP_AZURE_API_CLIENT_ID}/access_as_user`],
};
```

6.3 Wrap the app and attach tokens to API calls

```
// index.jsx
import { PublicClientApplication } from "@azure/msal-browser";
import { MsalProvider } from "@azure/msal-react";
import { msalConfig } from "./authConfig";

const msalInstance = new PublicClientApplication(msalConfig);

root.render(
  <MsalProvider instance={msalInstance}>
    <App />
  </MsalProvider>
);

// apiClient.js -- attach bearer token to every request
async function callApi(path, options = {}) {
  const account = msalInstance.getActiveAccount();
  const { accessToken } = await msalInstance.acquireTokenSilent({
    ...apiRequest,
    account,
  });
  return fetch(`${API_BASE_URL}${path}`, {
    ...options,
    headers: { ...options.headers, Authorization: `Bearer ${accessToken}` },
  });
}
```

6.4 Fetch permissions once, gate UI with a context + guard

```

// PermissionsContext.jsx
const PermissionsContext = createContext(null);

export function PermissionsProvider({ children }) {
  const [me, setMe] = useState(null);
  useEffect(() => {
    callApi("/api/me").then((r) => r.json()).then(setMe);
  }, []);
  return (
    <PermissionsContext.Provider value={me}>
      {children}
    </PermissionsContext.Provider>
  );
}

export function useHasFeature(featureKey, minLevel = "read") {
  const me = useContext(PermissionsContext);
  const levels = { read: 0, write: 1, admin: 2 };
  const granted = me?.permissions?.[featureKey];
  return granted !== undefined && levels[granted] >= levels[minLevel];
}

// Usage in a component
function Sidebar() {
  const canUseFormFactory = useHasFeature("form_factory", "write");
  const canUseConnectedApps = useHasFeature("connected_apps", "admin");
  return (
    <nav>
      {canUseFormFactory && <NavLink to="/form-factory">Form factory</NavLink>}
      {canUseConnectedApps && <NavLink to="/connected-apps">Connected apps</NavLink>}
    </nav>
  );
}

```

Frontend checks are UX only

Hiding a nav link does not stop a user from calling the API route directly. The FastAPI `require_permission` dependency is the actual security boundary; never skip it because “the frontend already checks.”

7. How middleware gatekeeping works end to end

This section restates the flow shown in the accompanying gatekeeping diagram, in implementation terms:

- **1. Request arrives** — React sends a request with `Authorization: Bearer <token>`.
- **2. `verify_token`** — FastAPI dependency validates the JWT's signature (against Azure AD's JWKS), issuer, audience, and expiry. Fails fast with 401 if invalid.
- **3. `resolve_identity`** — loads/creates the user record, resolves Azure AD groups (from token claim or Graph fallback) into internal roles and teams, using a short-TTL cache to avoid repeating this on every request.
- **4. `require_permission(feature, level)`** — checks whether any of the user's resolved roles grant the required access level on the requested feature. Fails with 403 if not.
- **5. Route handler executes** — only reached if all prior dependencies passed; scopes any data access to the user's team(s).

Because these are composable FastAPI dependencies (not a single monolithic middleware function), each route declares exactly the permission it needs, and the same three dependencies are reused everywhere — no per-route custom authorization logic.

8. Security considerations

- **Validate, don't trust, tokens.** Always verify signature, issuer, audience, and expiry server-side; never decode-and-trust a token without signature verification.
- **Store secrets in Secrets Manager**, not environment files or source control — API client secret/certificate, DB credentials.
- **HTTPS everywhere** — ALB terminates TLS with an ACM certificate; CloudFront in front of the S3-hosted SPA also enforces HTTPS.
- **Least-privilege IAM** for the ECS task role: only the permissions the backend actually needs (RDS access via IAM auth or Secrets Manager, CloudWatch Logs write, ElastiCache access).
- **Deny by default** for unmapped Azure AD groups — a user with no mapped role gets zero feature access, and this event should be logged for admin visibility.
- **Audit logging** — log login events, resolved role/team, and every 403 (permission-denied) event to CloudWatch Logs, with a retention/export policy to S3 if compliance requires longer retention.
- **Bounded staleness** — the Redis-cached identity resolution (15 min TTL suggested) means an Azure AD group removal takes up to that TTL to fully take effect, not immediate. Document this window; shorten the TTL if your compliance requirements need faster revocation.
- **Break-glass access** — keep a disabled-by-default local/manual login path for a small number of org admins in case Azure AD is unreachable; audit every use of it.

9. Implementation phases & rollout

Phase	Work	Exit criteria
1. Azure AD setup	Register SPA + API apps, configure groups/app-roles claim, grant API permissions.	Test user can obtain a token containing group claims via Postman/MSAL test harness.
2. Data model	Create Postgres tables (Section 4), seed roles/features, build minimal admin script to map groups → roles/teams.	Mapping table populated for at least one pilot team.
3. Backend auth	Implement <code>verify_token</code> , <code>resolve_identity</code> , <code>require_permission</code> ; wire <code>/api/me</code> .	A route protected by <code>require_permission</code> correctly returns 200 for allowed and 403 for denied test users.
4. Frontend auth	Integrate MSAL React, <code>PermissionsProvider</code> , <code>useHasFeature</code> guards on nav and pages.	Pilot users can log in via Azure AD and see only their permitted features.
5. AWS wiring	Deploy FastAPI to ECS Fargate behind ALB, SPA to S3/CloudFront, secrets in Secrets Manager, Redis cache.	End-to-end login + gated feature access works in a staging AWS environment.
6. Audit & logging	Wire CloudWatch logging for login/permission-denied events.	Logged events visible and queryable for a test login + a test 403.
7. Pilot rollout	Enable for one team/group; keep manual fallback available; monitor logs.	One week of pilot usage with no unexpected access issues.
8. Full rollout	Extend group→role/team mapping to remaining teams; decommission any legacy auth.	All users authenticate via Azure AD; legacy login path disabled or restricted to break-glass.

9.1 Testing checklist

- User with no mapped Azure AD group → logs in successfully but sees zero gated features (deny-by-default).
- User in an Editor-mapped group → can access Form Factory and Upload Contacts for their team, not Connected Apps.
- User in a Team Admin-mapped group → can access all three features, scoped to their team only.
- User removed from an Azure AD group → access revoked within the cache TTL window on next request.
- Token with >200 groups (or a simulated overage claim) → backend correctly falls back to Graph API.
- Expired/invalid/tampered token → 401, not 403 (distinguish authentication failure from authorization failure).
- Direct API call to a gated route without going through the UI → still enforces 403 for disallowed users.